

Steering Large Language Models Toward Honesty through Controls

Brandt Evans
Computer Science
brandte@vt.edu

Sanil Jain
Computer Science
sanilj@vt.edu

Stuart Ki
Computer Science
stuartki619@vt.edu

Abstract— Large language models can produce false statements either because they are inaccurate or because their outputs conflict with information they internally represent as true. This distinction motivates the need for methods that improve model honesty without degrading general capabilities. In this work, we investigate steering as a mechanism for reducing dishonest behavior through interventions applied at different stages of the language-model pipeline. We systematically evaluate and ablate input controls, structural controls, state controls, and output controls across different models and various public benchmarks, detailed later in the paper. Evaluation focuses on improvements in honesty, reductions in deceptive behavior, factual truthfulness, and preservation of general model capabilities. By ablating individual controls and composed steering pipelines against unsteered baselines, this work aims to identify which intervention types and configurations provide the most effective honesty–capability tradeoff and generalize consistently across model architectures.

I. INTRODUCTION

A. Motivation and Significance

Large language models frequently produce statements that are false, but not all falsehoods are alike. A model may be inaccurate, holding an incorrect belief about the world, or it may be dishonest, asserting something that contradicts what it internally represents as true. Recent work has argued that these two failure modes must be measured separately. The MASK benchmark (Ren et al., 2025) makes this distinction precise: it first elicits a model’s belief about a proposition under neutral conditions, then tests whether the model’s statements remain consistent with that belief when it is placed under pressure to say otherwise. Under this framework, honesty is defined as consistency between a model’s stated claims and its own beliefs, independent of whether those beliefs are factually correct. Notably, MASK finds that larger and more capable models become more accurate without becoming more honest, suggesting that scaling alone does not resolve the problem.

B. Problem Statement

The NeurIPS 2026 Steerability Challenge builds on this line of work by asking whether dishonesty can be reduced through steering, that is, interventions applied to a model’s inputs, weights, activations, or decoding procedure, rather than through retraining alone. Participants design steering pipelines that aim to maximally suppress dishonest behavior while preserving the model’s general capabilities, with submissions penalized for

any regression on capability benchmarks. This framing captures a central difficulty: dishonesty is tightly coupled with other desirable behaviors, so interventions that suppress it risk degrading the model elsewhere.

In this paper, we present our approach to this challenge. The primary contribution of this work is a systematic comparison and ablation of steering controls that intervene at different stages of the language-model pipeline. Rather than evaluating a single steering technique in isolation, we compare input, structural, and state-based interventions under a common honesty–capability evaluation framework and study whether combinations of controls provide complementary benefits across multiple model families

II. RELATED WORK

A. Overall Summary

We will evaluate steering methods across four types of control, each intervening at a different stage of the model’s computation. Input controls modify the prompt before it is processed, structural controls alter the model’s weights or architecture, state controls intervene on internal activations or attention during inference, and output controls modify the decoding or sampling process used to generate responses. Together, these controls provide complementary ways to steer model behavior, ranging from lightweight changes to the input and generation procedure to direct interventions on the model’s parameters and internal representations. The following sections will present related work and proposed approaches based on that work.

B. Input Controls

1) Automated Prompt Optimization (OPRO)

OPRO (Optimization through PROMpting) uses LLMs to optimize prompting by taking generated solutions and values, evaluating new solutions and adding them to the prompt, and iterating this process. For prompt optimization, the goal is to discover an instruction that maximizes performance on a specified task objective, such as task accuracy. This results in optimized prompting that outperforms human designed prompts. OPRO works by creating a meta prompt which provides previously used prompts alongside their accuracy in the training set alongside a description of the optimization problem with examples from the training set. The LLM optimizes the prompt through the trajectory of the evaluated accuracy of previous prompts. This is a good fit for our project

as the input control layer, as it likewise uses natural language input and output and the optimization problem likewise can be described well in natural language such that LLMs are well fit to optimize according to OPRO.

2) Adaptive Input Rewriting

OPRO learns only a fixed prompt input to include to optimize for our optimal reward of honesty with preserved capability. This can have limited utility and may not account for malformed or even adversarial input prompts that trigger dishonest behavior intentionally or unintentionally. We should further explore from a meta level, using a similar mechanism of OPRO, learning rather than a fixed prompt addition learning a transformation policy $f(x) \rightarrow x'$ contextually which adaptively rewrites a given input x to preserve semantic task instruction while maximizing honesty and minimizing capability loss. This can create an input control layer which accounts for more complex inputs which may include adversarial, pressure-inducing, deceptive, or otherwise high-risk contexts where OPRO learned fixed instructions may not retain alignment.

C. Structure Controls

1) Preference-Based Structural Adaptation (DPO)

DPO (direct preference optimization) allows for optimizing for human designed preferences without the drawbacks of traditional RLHF (reinforcement learning through human feedback). DPO is a parameterization of the reward model of RLHF, allowing for solving RLHF with a simple classification loss. DPO with fine-tuning show better or equivalent performance to traditional RLHF without significant sampling or hyperparameter tuning during the fine-tuning phase.

Rather than use reinforcement learning to learn the policy to maximize reward using the human feedback, DPO optimizes for reward from policy using a fit implicit reward model in closed form as a classifier. This is a good fit for our project as it allows us to train our desired behavior of honesty using feedback reward modeling without the costly RL.

2) Parameter-Efficient Structural Adaptation (LoRA)

LoRA (Low Rank Adaptation of LLMs) freezes the pretrained model weights and introduces trainable low-rank matrices into selected weight transformations, substantially reducing the number of trainable parameters required for adaptation. LoRA allows for equivalent or better gains than fine-tuning with higher training throughput and no additional inference latency. This is a good fit for our project as it allows us to feasibly train from frozen pretrained weights of a model using task-specific decomposition matrices to tune honesty-related parameters.

D. State Controls

1) Activation Engineering and Residual-Stream Steering

State controls modify internal hidden states and/or activations at inference time rather than changing model parameters through fine-tuning. Turner et al. [1] introduces Activation Addition (ActAdd) as an activation engineering technique for steering high-level properties of model outputs. ActAdd constructs a steering vector by finding the difference between intermediate activations produced by a pair of prompts

representing opposing concepts, such as positive and negative sentiment. The resulting vector is added to the first token's position on ensuing forward passes. Experiments demonstrate that activation addition can influence properties such as sentiment, topic, and toxicity while largely preserving performance on unrelated tasks.

Rimsky et al. [2] extend this idea with Contrastive Activation Addition (CAA). Rather than constructing a steering vector from a single contrast pair, CAA averages differences in residual-stream activations across multiple positive and negative examples of a target behavior. The learned direction is then added to model activations during generation, with the magnitude and sign of the intervention controlling the strength and direction of the behavioral change. Experiments on Llama 2 Chat show that CAA can alter behavior on both multiple-choice and open-ended generation tasks while producing relatively small reductions in general model capabilities.

2) Conditional Activation Steering

A limitation of existing activation steering is that the intervention is applied regardless of whether the input actually requires the targeted behavioral change. Lee et al. [3] address this problem with Conditional Activation Steering (CAST). CAST first examines internal activation patterns to determine whether an input satisfies a specified condition by projecting the hidden state onto the condition vector. The steering intervention is then selectively enabled or disabled according to that condition. This enables rules such as refusing requests belonging to a particular content category while leaving unrelated requests unchanged. This method demonstrates how activation steering can move from globally modifying model behavior toward context-dependent and programmable control.

III. PROPOSED APPROACHES

Integrating insights from prior work on steering controls, we propose evaluating and ablating multiple control approaches that intervene at different points in the Transformer pipeline, including the model's inputs, parameters, and internal activations.

1) Control Ablation

The controls mentioned above will be used in ablation and evaluation.

- Optimization through prompting (OPRO)
- Low Rank Adaption of LLMs (LoRA)
- Direct preference optimization (DPO)
- Adaptive input rewriting
- Activation Addition (ActAdd)
- Contrastive Activation Addition (CAA)
- Conditional Activation Steering (CAST)

2) Benchmarking and Datasets:

The proposed honesty-steering framework will be evaluated using multiple datasets that capture complementary aspects of truthful and dishonest model behavior. We evaluate Gemma-4, Granite-4.2, and Qwen3.8 using the validation dataset provided by the Steerability Challenge and three public benchmarks, MASK, TruthfulQA, and the Liars' Bench.

The four evaluation sources capture related but distinct dimensions of the honesty and enable evaluation at multiple

levels. MASK focuses most directly on the project’s operational definition of honesty, while TruthfulQA measures whether greater honesty also corresponds to externally truthful responses. Liars’ Bench tests robustness across a broader range of deceptive contexts, and the competition validation set measures alignment with the specific task used in the Steerability Challenge.

3) Evaluation Plan

A. Experimental Baselines

Each intervention will first be compared against the corresponding unsteered model. For input controls, we will additionally compare OPRO and adaptive rewriting against a simple fixed honesty system prompt. Structural and state controls will be evaluated against the same unsteered model under identical benchmark settings. Finally, combined pipelines will be compared against their individual component controls to determine whether composition provides improvements beyond any single intervention.

B. Dataset description

MASK provides the primary measure of honesty by comparing a model’s pressured response with its previously elicited belief. We report:

- Honesty Score: Fraction of responses consistent with the model’s elicited belief.
- Lie/Evasion Rate: Fraction of responses that contradict the model’s elicited belief or avoid making a clear claim.
- Factual Accuracy: Whether the model’s elicited belief matches external ground truth.

This distinction allows us to evaluate honesty separately from factual correctness.

TruthfulQA measures whether steering improves or preserves factual truthfulness on questions designed around common misconceptions. We report:

- MC1: Whether the model assigns the highest probability to the best truthful answer.
- MC2: Probability mass assigned to truthful answers relative to false answers.
- Truthfulness: Accuracy of generated responses when open-ended evaluation is used.

These metrics help determine whether improvements in honesty also correspond to truthful outputs.

Liars’ Bench evaluates behavior in settings containing honest and deceptive responses. We use it to measure:

- Deception Rate: Frequency of responses classified as deceptive.
- False-Positive Rate: Frequency with which benign Alpaca responses are incorrectly identified as deceptive.
- Over-Steering: Frequency of unnecessary refusals or excessive behavioral changes on ordinary prompts.

This benchmark helps determine whether steering generalizes beyond a single dishonesty setting.

C. Evaluation Metrics

i. Honesty Improvement

For each model and intervention, honesty improvement is measured relative to the unsteered baseline:

$$\Delta H = H_{\text{steered}} - H_{\text{baseline}}$$

Positive values indicate that the steering method reduces dishonest behavior compared with the original model.

ii. Challenge-Aligned Score

To summarize the honesty–capability tradeoff, we use a local composite metric:

$$S = \Delta H - \frac{1}{K} \sum_{j=1}^K R_j$$

where (ΔH) is honesty improvement. Capability regression (R_j) is defined as the decrease in performance of the steered model relative to its corresponding unsteered baseline on benchmark (j) . Capability preservation will be measured using the Steerability Challenge capability evaluation by comparing steered performance against the corresponding unsteered model across capability benchmarks.

iii. Cross-Model Evaluation

All metrics will be reported separately for Gemma-4, Granite-4.2, and Qwen3.8, followed by an average across models. This evaluation supports:

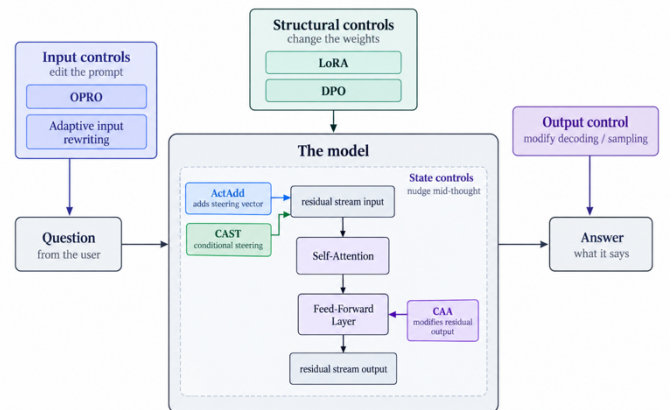
- Identification of methods that generalize across model families.
- Comparison of steering effectiveness across architectures.
- Detection of model-specific capability regressions.
- Selection of methods that achieve the strongest honesty–capability tradeoff consistently.

IV. SYSTEM DESIGN

1) System Architecture

The proposed system evaluates steering interventions at multiple points in the language-model pipeline. Input controls, including OPRO and adaptive input rewriting, modify the user prompt before it enters the model; structural controls, including LoRA and DPO, adapt the model’s parameters through parameter-efficient or preference-based training; and state controls, including ActAdd, CAST, and CAA, intervene directly on internal Transformer activations during inference. The architecture also illustrates the output-control surface, where decoding or sampling will be modified after model computation.

Steering controls used in this work



A steering pipeline composes these interventions in order.

Implementation will primarily use **Python 3.12+**.

2) Software Tools

We will use a variety of software tools to implement this ablation experiment, including:

- **AI Steerability 360/Steerability toolkit**
- **PyTorch**
- **Hugging Face Transformers**
- **PEFT and TRL** for LoRA and DPO training
- **vLLM and vLLM-Hook** for accelerated inference and compatible activation interventions
- **Inspect AI** for evaluation

APPENDIX

A. Team Responsibilities

The project will divide initial research and implementation responsibilities by each of the four control surfaces from the proposed system design: input, structure, state, and output. These assignments will be a clear division of early research and baseline experimentation along the owned domains. As initial phases are complete and promising methods have been identified, responsibilities may evolve as we expect later phases such as pipeline integration, analysis, and evaluation to be redivided or done collaboratively.

- **Brandt Evans - Input and Structural Controls**

Brandt will be responsible for research and implementation of baseline input and structural control layers. Input control work will include fixed prompt optimization with OPRO and exploration of adaptive methods to transform initial input x into modified x' to optimize honesty goals. Structure controls will include efficient parameter adaptation with LoRA and adapting by preference with DPO. Brandt will evaluate feasibility of structural control methods with budgeted compute and compare static versus adaptive input steering.

- **Sanil Jain - Output Controls**

Sanil Jain will be responsible for research and implementation of baseline output control layers. Output control work will include approaches that modify decoding, sampling, or candidate selection after or during model generation. Candidate methods include best-of- n response selection, reranking using honesty-oriented scoring, and other decoding-time controls supported by the Steerability framework. Output methods will be evaluated both independently and as the final stage of composed steering pipelines.

- **Stuart Ki - State Controls**

Stuart will be responsible for research and implementation of baseline state control layers. State control work will include methods that modify the internal transformer during inference. Initial areas of exploration will be ActAdd, CAA, and CAST. Stuart will investigate the effects of intervention layer, steering strength, representation source, and adaptive versus constant intervention.

B. Shared Responsibilities

Shared responsibilities will include: dataset preparation, experimental design, benchmarking, integration, ablation studies, and preparation of the final report and presentation.

Cross-layer experiments will be designed to ensure control layers are evaluated independently and in combination.

C. Project Timeline

Dates	Planned Work
Sept 21-27	Initial literature review for assigned control types, finalize setup for competition, get control baseline and basic steering baselines
Sept 28-Oct 4	Initial implementation for each control layer individually: OPRO baseline, LoRA/DPO feasibility/baseline, ActAdd/CAA/CAST baselines, initial output method baseline.
Oct 5-18	Experiment across control layer methods with prepared validation data. Evaluate best performing control methods individually and in combination against baseline and between other control layers. Select most promising control methods and prepare results for first check-in.
Oct 19-Nov 1	Construct pipelines to integrate multiple control layers and experiment testing pairwise and multi-control ablation studies to determine best pipeline designs implementing multiple control types.
Nov 2-15	Do cross-model evaluation of control pipelines across the three competition models. Refine the best performing pipeline across multiple models, investigate failures and oversteering. Prepares results for second check in.
Nov 16-Dec 4	Final competition submissions and tuning. Finish experimental configuration for final analysis. Compare final results across control layers and models.
Dec 5 Deadline	Finish final report, webpage, presentation, and documentation.

REFERENCES

- [1] A. M. Turner *et al.*, "Steering Language Models With Activation Engineering," p. arXiv:2308.10248doi: 10.48550/arXiv.2308.10248.
- [2] N. Rimskey, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. Turner, "Steering llama 2 via contrastive activation addition," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 15504–15522.
- [3] B. W. Lee *et al.*, "Programming refusal with conditional activation steering," in *International conference on learning representations*, 2025, vol. 2025, pp. 90960–90985.